Every number, in one place

Each figure below names the script that produces it. Run that script and you get the number back. Nothing here is quoted from memory.

---

1. Headline result

Grouped 5-fold cross-validation, 20 repeats, over the 600 labelled rows (300 candidates x 2 phases). Source: `predict.py`, stored in `model_card.json`.

metric	model	baseline
multi-class log loss	**0.906**	1.096 (guessing the base rates)
accuracy	**0.558**	0.355 (always guess the commonest class)
macro F1	**0.560**	0.175
macro AUC	**0.740**	0.500
priority ranking AUC	**0.811**	0.500

By phase, showing the model does move on later evidence:

	log loss	accuracy	macro AUC	priority AUC
T0 (initial evidence)	0.931	0.520	0.718	0.794
T1 (after updates)	**0.880**	**0.597**	**0.762**	**0.829**

Honest version. Those figures share the 300 labels with roughly forty design comparisons, which inflates them. Nested selection — choosing the design inside a training split, then scoring once on unseen candidates — gives **0.913 log loss, 0.549 accuracy, 0.737 macro AUC**. Selection bias is therefore 0.008. Source: `audit_honest.py` / `audit_honest_output.txt`.

---

2. The queue: what a reviewer actually opens

The operational figures. Measured on the 300 held-out cases at T0, which is the realistic "first pass over a backlog" scenario. Source: `audit_capacity.py`, `audit_topdecile.py`.

Of all 300 cases: 90 truly worth reviewing, 105 truly "cannot tell", 105 truly not worth it.

slice of queue	cases	worth reviewing	cannot tell	not worth it
top 5%	15	12 (80%)	3 (20%)	**0 (0%)**
top 10%	**30**	**23 (77%)**	5 (17%)	**2 (7%)**
top 20%	60	37 (62%)	15 (25%)	8 (13%)
top 50%	150	72 (48%)	54 (36%)	24 (16%)
whole pile, unsorted	300	90 (30%)	105 (35%)	105 (35%)

Reading of the top 10%:

- **77%** is the share genuinely worth reviewing. It does **not** include "cannot tell".
- **93%** (28 of 30) need a human at all, since "cannot tell" also requires a person.
- **7%** (2 of 30) are genuinely wasted opens, against 35% for an unsorted pile.
- **2.6x** more worthwhile work found per file opened than working the list in any order.
- Half of all worthwhile cases sit in the top 28% of the queue; 80% of them in the top 50%.

Caveat that must be stated with it: the top-10% figure rests on 30 cases. The 95% interval on 77% runs from about **60% to 89%**. The direction is solid, the exact number is not.

---

3. Why accuracy is 56%, and the ceiling

Source: `audit_ceiling.py` / `audit_ceiling_output.txt`.

probe	result	what it means
a model allowed to memorise the 300 answers	**100%** training accuracy	the features are expressive enough; the limit is generalisation, not capacity
the ground truth graded twice, three months apart	agrees with itself **77.7%** of the time	the official answer changes for 22.3% of candidates, so ~78% is the ceiling for anyone
nearest-neighbour cases in feature space	agree on their label **49%** of the time	high intrinsic noise. Caveat: those neighbours sit at median distance 4.3 vs 8.8 for random pairs, so they are about twice as close as chance, not identical
re-weighting the decision rule purely for accuracy	**+0.008**	and that was tuned on the same 300 cases, so the honest gain is smaller; it costs calibration

If any solution reports 90% on this data, it has leaked labels or memorised the development set. The target itself is not stable enough to support it.

The same predictions, framed other ways

question	result
both the model and the truth commit to a decided verdict	**88.5%** (239 of 270)
warranted vs everything else	77.7%
predictions landing within one band of the truth	94.8%
predictions at the opposite end of the scale from the truth	5.2%

The 88.5% figure is only honest when the exclusion is said in the same breath.

Confusion matrix, rows true / columns predicted, order not-warranted / cannot-tell / warranted:

```
[[136  61  16]
 [ 61  96  51]
 [ 15  61 103]]
```

Calibration

Predicted probability against how often it actually happened, in quintiles:

quintile	p(warranted) predicted / observed	p(not warranted) predicted / observed
1	0.05 / 0.01	0.07 / 0.11
2	0.13 / 0.17	0.17 / 0.14
3	0.24 / 0.22	0.30 / 0.33
4	0.41 / 0.42	0.47 / 0.42
5	0.65 / 0.67	0.73 / 0.78

No post-hoc recalibration was applied.

---

4. Model comparison

Every family on identical grouped folds, each with a hyperparameter sweep. The boosters were given two extra advantages: the observed tag state as a genuine nine-level categorical (which only they can use), and the option of running under the same ordinal decomposition the winner uses. Best configuration per family shown. Source: `benchmark_models.py` / `benchmark_output.txt`.

model	log loss	accuracy	macro F1	macro AUC
ordinal logistic (shipped)	**0.907**	**0.554**	**0.556**	**0.741**
multinomial logistic	0.924	0.538	0.532	0.725
random forest	0.951	0.527	0.523	0.703
CatBoost (with categorical)	0.964	0.510	0.508	0.705
XGBoost	0.983	0.507	0.504	0.695
hist gradient boosting	0.998	0.503	0.501	0.691
LightGBM	1.047	0.496	0.495	0.682
class prior (guessing)	*1.096*	*0.355*	*0.175*	*0.500*
LLM classifier (Claude Haiku 4.5)	**1.151**	0.497	—	0.659

Three things worth noting:

- **Every tree ensemble loses to plain logistic regression.** 600 rows against 41 features is far less data than boosting wants; the signal is a smooth monotone function that trees can only approximate as a staircase; and the ordinal constraint is information a multiclass tree cannot represent.
- **The gap is calibration, not discrimination.** CatBoost trails on accuracy by about four points but on log loss by 0.06.
- **Handing the boosters the ordinal structure made them worse** (XGBoost 0.983 -> 1.037, CatBoost 0.964 -> 1.145): two models where there was one, and at this sample size the extra variance costs more than the structure gains.

The LLM classifier scored worse than guessing the base rates. It was given the same evidence summary and the same three-way task on the same held-out cases. Source: `llm_baseline.py` / `llm_baseline_output.txt`.

---

5. Identity resolution

The hard part of the problem. Source: `diagnostics.py`, `audit_alias.py`.

The corruption pattern. Each candidate has four records per feed. Clustering owner names inside each vehicle (grouped by `vehicle_ref`, which is independent of names) gives a 3 + 1 split for 98% of vehicles: three lightly corrupted copies of one name, and one name replaced outright. Grouping the licence feed by date of birth reproduces it.

The replaced name is a persistent alias, not fresh noise.

	replaced name found in feed	recombined-name control
address	43.3%	3.8%
external	43.6%	3.9%
licence	31.9%	3.8%

An 11x enrichment over chance. It is per-candidate rather than a shared pool: 4,906 of 5,278 distinct aliases point at exactly one vehicle, and alias-carrying address rows are unlinked by name matching **72.6%** of the time against a **19.5%** baseline. 8,067 aliases recovered, covering two thirds of candidates.

feed	linked	of	via alias
address	41,567	48,121	2,827
licence	40,855	48,124	2,129
external	41,508	48,116	2,877
work	24,131	24,131	0
title	47,581	48,108	via `vehicle_ref`
T1 updates	21,949	24,000	1,856

---

6. Where the signal is

Spearman rho against the ordered T0 label, recency-weighted at a 150-day half-life. Source: `diagnostics.py`, `audit_unused.py`.

feed	rho
current address only (blank end date)	**+0.41**
title	+0.37
address, all rows	+0.32
external	+0.29
licence	+0.15
work location	**-0.07**
four feeds pooled	**+0.53**
observed tag state (independent, additive)	+0.11

Two findings behind the feature set: the **work-location feed carries no signal at all** and is excluded; and the **single current-address record out-predicts the entire address feed**, so it gets its own feature block.

Aggregation methods compared:

aggregation	rho
exponential decay, 150-day half-life	**+0.52**
rank decay	+0.51
newest record per feed, averaged	+0.48
hard window, last 180 days	+0.46
change-point, post-segment mean	+0.41
linear-in-time extrapolation	+0.39

7. Would more labelled data help?

Source: `audit_learning_curve.py`. Held-out set fixed at 75 candidates throughout.

training candidates	40	70	105	145	190	225
log loss	0.976	0.943	0.928	0.922	0.916	0.914

The marginal value of 50 extra labelled candidates falls from **+0.054** early on to **+0.003** at the top of the range. **More labels would not move this much.** The binding constraint is noise in the evidence, not the size of the label set.

8. What was tried and rejected

Each judged on the same grouped cross-validation. Source: `experiments.py`, `audit_features.py`, `audit_ceiling.py`.

idea	result
per-feed 90- and 365-day half-lives	+0.001, no gain
splitting T1 updates by `record_action`	+0.000, noise
non-linear terms on the direction index	+0.002, worse
a "T1 title changed state" flag	-0.002, inside the noise
observed tag x direction interaction	+0.002, worse
explicit old-versus-new trend per feed	+0.006, worse
self-training on the ~11,700 unlabelled cases	worse at every confidence threshold and every pseudo-label weight, from +0.002 to +0.165
forcing unclaimed rows on via a capacity constraint	coverage rose to ~99% but three feeds got weaker
recovering the replaced address row from its timeline gap	impossible: consecutive address records chain end-to-start in 3 of 8,322 cases
XGBoost, LightGBM, CatBoost, random forest	all lose to the ordinal logistic (section 4)
an LLM classifier	worse than guessing (section 4)
tuning the decision rule for accuracy	+0.008, and it costs calibration

9. Submission integrity

- `case_predictions.csv` — 24,000 rows, the template's exact schema and row order.
- Every row's three probabilities sum to **exactly 1** in decimal, checked by parsing the written file rather than the in-memory floats.
- `validate.py` runs 11 format checks; all pass.
- `predict.py` is deterministic: from a clean directory with the data at any path, it reproduces `case_predictions.csv` **byte for byte**.
- The supplied package was verified against the organisers' own `Package_Manifest.csv`: all 15 files checksum-match. One listed file, `finalize_and_audit_oos_participant_package.py`, was not shipped.